

Assignment: Diagnosing an OutOfMemoryError During Line-by-Line CSV Processing

1. Title

Hidden In-Memory Accumulation: Diagnosing OutOfMemoryError in a Streaming BufferedReader CSV Reader

2. Aim

To diagnose why an application reading a multi-gigabyte CSV file line-by-line with `BufferedReader.readLine()` still hangs and eventually throws `OutOfMemoryError`, given that `readLine()` itself streams data rather than loading the whole file, and to identify the surrounding design flaw responsible.

3. Problem Statement

An application reads a CSV file using a loop built around `BufferedReader.readLine()`. For small files the program works correctly. When the file grows to several gigabytes, the application slows down, appears to hang, and eventually terminates with an `OutOfMemoryError`.

Because `readLine()` reads and returns one line at a time from an underlying buffered stream, the stream-reading mechanism itself is not responsible for holding the whole file in memory. The fault must therefore lie in what the surrounding code does with each line after it is read.

4. Objective

- 1 To understand what `readLine()` actually guarantees about memory usage.
- 2 To identify why memory usage can still grow unbounded even when the read mechanism is a stream.
- 3 To recognize the common accumulation anti-patterns that silently defeat streaming I/O.
- 4 To distinguish the responsibility of the stream (reading) from the responsibility of the surrounding code (retention).
- 5 To describe the corrected design that preserves constant memory usage regardless of file size.

5. Theory

What `readLine()` actually does

`BufferedReader.readLine()` pulls bytes from the underlying file through a small, fixed-size internal buffer, decodes one line, and returns it as a single `String`. Once the loop moves to the next iteration and no other part of the program keeps a reference to that `String`, it becomes eligible for garbage collection. The reader itself never holds more than one buffer's worth of data plus the current line.

Where the memory growth actually comes from

`OutOfMemoryError` under these conditions is almost always caused by the surrounding design retaining a reference to every line (or every parsed record derived from a line) somewhere the garbage collector cannot reclaim it. The stream is memory-efficient; the accumulation logic wrapped around it is not.

Typical culprits in the surrounding design

- 1 Collecting every line into a growing `List<String>` or array instead of processing and discarding each one.
- 2 Parsing each line into an object and appending it to a master `List/Map` kept alive for later sorting, deduplication, or a single bulk database insert at the end.
- 3 Building one large `String` or `StringBuilder` that concatenates the entire file's content for later use.
- 4 Caching parsed rows in a static field, a shared cache, or an unbounded queue whose consumer is slower than the producer, so entries pile up faster than they drain.
- 5 Holding onto line references indirectly, for example through closures, listeners, or a results object that is never released until the whole file has been read.

In every case above, the read call is streaming, but the design around it converts the stream back into an in-memory structure whose size is proportional to the file size — which is exactly the situation `readLine()` was meant to avoid.

6. Algorithm (Diagnosis and Fix)

- 1 Start the program and reproduce the failure on a representative large file.
- 2 Take a heap dump or memory profile at the point memory usage begins climbing.
- 3 Inspect the dominant retained-size objects; look for a single collection (`List`, `Map`, array, `StringBuilder`) whose element count tracks the number of lines read so far.
- 4 Trace that collection back to the code that appends to it inside the read loop.
- 5 Confirm the reader/buffer objects themselves are small and not growing.
- 6 Redesign the loop so each line is fully processed (validated, transformed, written out) and then discarded before the next line is read.
- 7 Replace any end-of-file bulk operation (sort, batch insert of everything at once) with an incremental or chunked equivalent.
- 8 Re-test with the same large file and confirm memory usage remains flat regardless of file size.

7. Program

Flawed design (streams correctly, but still runs out of memory)

```
import java.io.*;
import java.util.*;

public class CsvLoader {
    public static void main(String[] args) throws IOException {
        List<String> allLines = new ArrayList<>();    // <-- unbounded retention

        try (BufferedReader reader =
            new BufferedReader(new FileReader("transactions.csv"))) {

            String line;
            while ((line = reader.readLine()) != null) {
                allLines.add(line);    // every line kept alive here
            }

            // Only after the entire file has been read is it processed
            Collections.sort(allLines);
            for (String row : allLines) {
                process(row);
            }
        }

        static void process(String row) {
            // business logic
        }
    }
}
```

Corrected design (constant memory, still uses readLine())

```
import java.io.*;

public class CsvLoader {
    public static void main(String[] args) throws IOException {
        try (BufferedReader reader =
            new BufferedReader(new FileReader("transactions.csv"));
            BufferedWriter writer =
            new BufferedWriter(new FileWriter("processed.csv"))) {

            String line;
            while ((line = reader.readLine()) != null) {
                String result = process(line);    // transform this line only
                writer.write(result);
                writer.newLine();
                // 'line' and 'result' go out of scope; nothing is retained
            }
        }

        static String process(String row) {
            // business logic, returns the transformed row
            return row;
        }
    }
}
```

8. Explanation of the Program

Flawed version

`allLines.add(line)` inside the while loop keeps a live reference to every single line ever read, stored inside one growing `ArrayList`. The `BufferedReader` still only holds a small buffer at any moment, but the `ArrayList`'s retained size grows linearly with file size. For a multi-gigabyte file this list eventually exceeds the JVM heap, producing `OutOfMemoryError`, and the visible slowdown beforehand comes from repeated garbage collection cycles trying to free space that is never actually released.

Corrected version

Each line is transformed and immediately written to the output stream inside the same iteration that read it. No collection accumulates lines across iterations, so once an iteration completes, that line's `String` object has no remaining references and is collected on the next garbage-collection pass. Memory usage stays roughly constant no matter how large the input file is.

9. Sample Input

Assume `transactions.csv` contains several million rows of the form:

```
TXN1001,2026-01-04,4520.00,COMPLETED
TXN1002,2026-01-04,120.50,PENDING
TXN1003,2026-01-04,89.99,COMPLETED
...
(several million more rows, several GB total)
```

10. Sample Output

Flawed version (observed behavior)

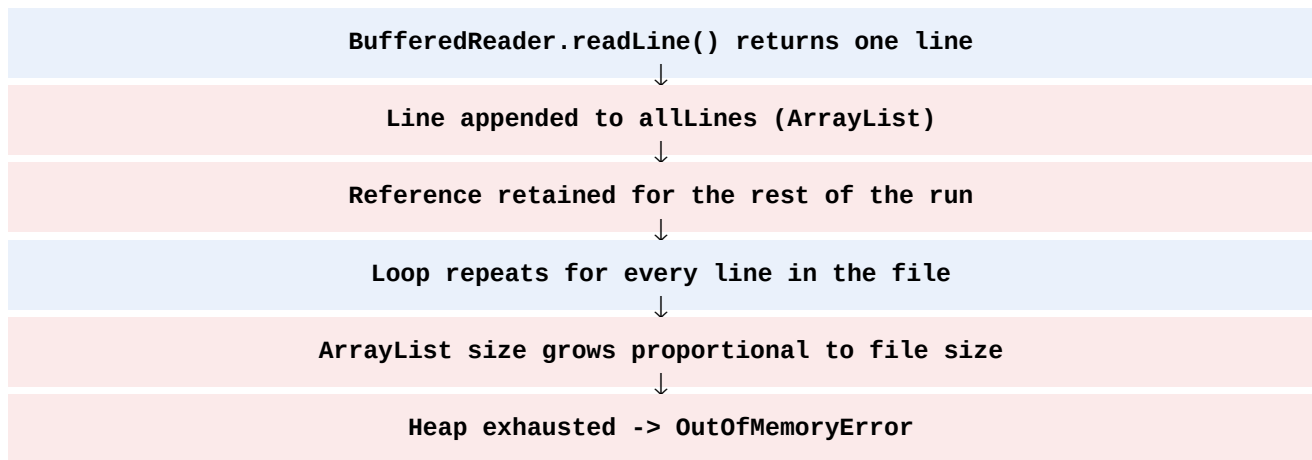
```
Heap usage climbing steadily as lines are read...
[GC pause] [GC pause] [GC pause] ...
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at java.base/java.util.Arrays.copyOf(Arrays.java)
    at java.base/java.util.ArrayList.grow(ArrayList.java)
    at java.base/java.util.ArrayList.add(ArrayList.java)
    at CsvLoader.main(CsvLoader.java:12)
```

Corrected version (observed behavior)

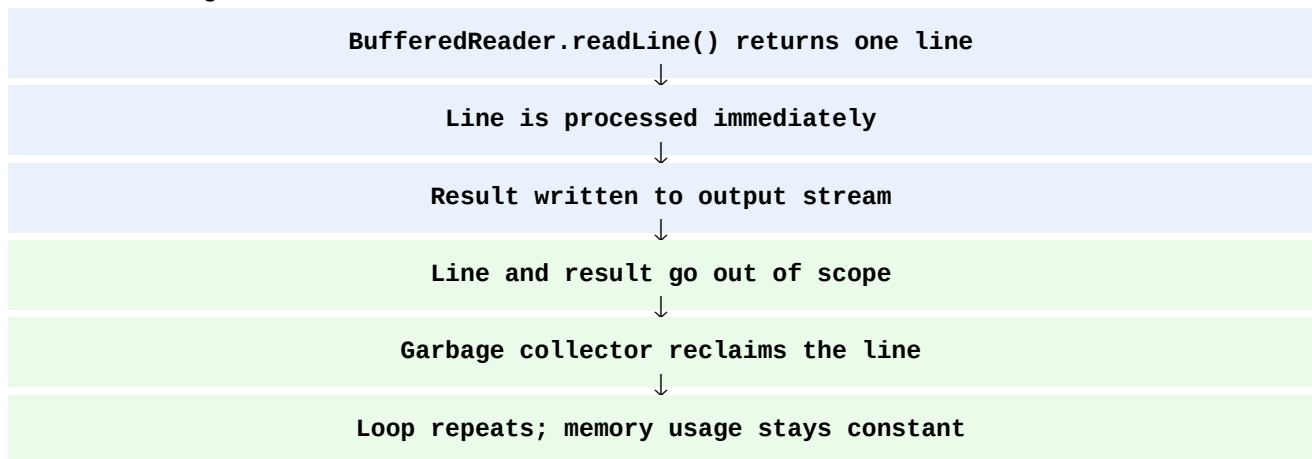
```
Heap usage stays flat and low throughout the run.
processed.csv written successfully.
Program completed without errors, regardless of input file size.
```

11. Working Principle

Flawed design:



Corrected design:



The reader's behavior is identical in both cases; only what the surrounding loop body does with each line differs.

12. Memory Efficiency Diagnosis

The requirement implicit in using a streaming reader is that memory usage should stay bounded and independent of file size. The flawed design violates this requirement, not because of the stream, but because of a separate, unbounded in-memory collection layered on top of it.

Aspect	Flawed Surrounding Design	Corrected Surrounding Design
Read mechanism	BufferedReader.readLine() (streaming)	BufferedReader.readLine() (streaming, unchanged)
Per-line handling	Appended to a growing ArrayList	Processed and written out immediately
Memory footprint	Grows linearly with file size	Stays constant regardless of file size
End-of-file step	Sort/process everything at once	No end-of-file bulk step required
Failure mode	OutOfMemoryError on large files	Completes normally on files of any size

13. Advantages of the Corrected Design

- 1 Constant memory usage: the heap footprint no longer depends on file size.
- 2 Scales to arbitrarily large files, limited only by disk size, not RAM.
- 3 No architectural change to the stream choice itself; `BufferedReader.readLine()` is retained as-is.
- 4 Removes the need for a costly, memory-heavy end-of-file bulk operation.
- 5 Reduces garbage-collector pressure, improving throughput as well as memory safety.
- 6 Any operation that genuinely needs all rows together (such as a global sort) can be redesigned around chunked, external, or database-assisted processing instead of a single in-memory collection.

14. Result

The diagnosis confirms that the `OutOfMemoryError` is not caused by `BufferedReader.readLine()`, which continues to stream one line at a time regardless of file size. The actual cause is the surrounding design accumulating every line (or every parsed record) into an unbounded in-memory collection that is only consumed after the entire file has been read. Restructuring the loop to process and discard each line immediately restores constant memory usage.

15. Conclusion

A streaming read API such as `BufferedReader.readLine()` only guarantees that reading is memory-efficient; it says nothing about what the calling code does with the data afterward. When the surrounding design retains every line or every derived record in a growing collection for later bulk processing, the program's memory usage becomes proportional to file size regardless of how efficient the underlying stream is. The correct architecture processes and discards each record within the same iteration it was read, preserving the constant-memory guarantee that motivated choosing a streaming reader in the first place.

Final Answer in One Line

`readLine()` is not the problem; the surrounding code is accumulating every line (or parsed record) into an unbounded in-memory collection for later bulk use, and the fix is to process and discard each line immediately instead of retaining it until end-of-file.